# Сценарий выступления: «Пиксель-арт из твоего фото»

**Для преподавателя.** Полный текст, который можно говорить, слайд за слайдом. Читать дословно не обязательно — это опора: своими словами всегда лучше.

| | |
|---|---|
| **Длительность** | 90 минут (тайминг совпадает с конспектом) |
| **Презентации** | `slides/presentation_shct.pdf` или `slides/presentation_mastersam.pdf` — слайды одинаковые, отличаются только оформлением, фото и последним слайдом |
| **Возраст** | 11–17 лет, смешанная группа |

**Обозначения:**

- 🗣️ — текст, который говорите вслух;
- ❓ — вопрос группе, дальше в скобках — ожидаемый ответ;
- 🖐️ — действие (показать, переключить, запустить);
- ⏱️ — сколько времени на слайд.

> **Перед началом:** откройте презентацию, запустите на своём компьютере `main.py` и приложение `app/pixel_paint.py` один раз «вхолостую», чтобы не ждать загрузки при детях. Подготовьте 2–3 пиксельных ролика для слайда 9 и пару фото для демонстрации.

---

## Слайд 1. Титульный — «Пиксель-арт из твоего фото»

⏱️ 2 минуты

🗣️ Привет! Меня зовут ___, и сегодня мы с вами за полтора часа сделаем одну вещь, которая звучит как магия: возьмём обычную фотографию — вашу, вашего кота, любимого героя — и превратим её в пиксель-арт. Как в старых играх: Марио, ранний Minecraft, тетрис.

🗣️ Посмотрите на слайд. *(В версии МАСТЕРСАМ: слева сверху фото, внизу то же фото в пикселях. В версии ШЦТ: кружок с пиксельным котом.)* Вот это сделал не художник и не фотошоп. Это сделала программа на Python из примерно десяти строчек. К концу занятия у каждого из вас будет такая же картинка из своего фото — и ещё раскраска, которую можно распечатать.

❓ Поднимите руку, кто уже когда-нибудь писал код? На любом языке, хоть в Scratch. *(Посмотрите на зал — это поможет понять, как распределить внимание.)*

🗣️ Отлично. Если вы никогда не программировали — это нормально, сегодня всё будет понятно с нуля. А тем, кто уже умеет, я подготовил задания посложнее — скучно не будет.

---

## Слайд 2. «Что мы сегодня сделаем»

⏱ 2 минуты

🗣 План у нас из четырёх шагов.

🗣 **Первое — узнаем, из чего состоят картинки.** Пиксели, разрешение, что такое RGB. Без этого непонятно, что именно мы будем менять в коде.

🗣 **Второе — посмотрим, что за пиксельные видео сейчас в TikTok.** Вы наверняка их видели, а сегодня поймёте, как они сделаны.

🗣 **Третье — напишем код на Python**, который превращает фото в пиксель-арт.

🗣 **И четвёртое — соберём из этого кода свою библиотеку** и раскрасим результат в приложении.

🗣 Начнём с самого главного вопроса.

---

## Слайд 3. «Картинка — это числа»

⏱ 3 минуты

❓ Как вы думаете, как компьютер видит фотографию? Вот на экране котик *(в версии МАСТЕРСАМ — люди на фото)*. Компьютер понимает, что это котик?

*(Ждём ответы. Обычно говорят «видит цвета», «нули и единицы», «понимает через нейросеть».)*

🗣 Хорошие версии. Но на самом деле компьютер **вообще не видит котика**. Для него картинка — это таблица чисел. Огромная таблица.

🖐 Покажите на сетку 8×8 слева.

🗣 Смотрите: вот мы взяли маленький кусочек фото и сильно его увеличили. Получилась сетка 8 на 8 клеточек. В каждой клеточке записан **ровно один цвет**. Не узор, не градиент — один цвет.

🗣 А цвет компьютер тоже хранит числами. Справа написано: клеточка в четвёртом ряду, четвёртом столбце — это три числа: **(74, 56, 41)** *(в версии МАСТЕРСАМ — (58, 44, 24))*. Что значат эти три числа, разберём через пару слайдов. Пока запомните главное: **любая картинка в компьютере — это сетка, и в каждой клетке записан цвет числами.**

🗣 А раз картинка — это числа, значит, мы можем менять её кодом. Как числа в калькуляторе. Это и есть основа всего, что мы сегодня сделаем.

---

## Слайд 4. «Что такое пиксель»

⏱ 3 минуты

❓ Кто знает, как называется вот эта маленькая клеточка с одним цветом? *(Пиксель.)*

🗣 Правильно, пиксель. Само слово — сокращение от английского *picture element*, «элемент картинки».

🖐 Покажите три картинки слева направо.

🗣 Смотрите, что будет, если приближать фото. Первая картинка — фото целиком, всё гладко и красиво. Вторая — приблизили в 4 раза: ещё нормально, детали видны. Третья — та же картинка, но

маленькая, 48 на 48 пикселей, и мы её растянули. И вот теперь видны **квадратики**. Каждый квадратик — один пиксель.

🗣️ Внизу определение: **пиксель — самый маленький кусочек картинки. Один квадратик — один цвет.** Меньше пикселя на картинке ничего нет. Даже если вы приблизите фото в телефоне до предела, в какой-то момент упрётесь в квадратики.

❓ Кстати, а у экрана вашего телефона тоже есть пиксели? *(Да.)* Если посмотреть на экран через каплю воды — она работает как лупа, — можно их увидеть. Попробуйте дома.

---

## Слайд 5. «Разрешение картинки»

⏱️ 3 минуты

🖐️ Покажите три картинки.

❓ Какая из трёх картинок самая чёткая? И почему? *(Правая — в ней больше пикселей.)*

🗣️ Верно. Это одно и то же фото, но с разным количеством пикселей. Левая — 16 на 16, всего 256 пикселей, и котика уже почти не узнать. Средняя — 48 на 48, это 2304 пикселя. Правая — 192 на 192, почти 37 тысяч пикселей.

🗣️ Вот это и называется **разрешение** — сколько пикселей в ширину умножить на сколько в высоту.

🗣️ Вы постоянно встречаете это слово. **Full HD** — это 1920 на 1080, около 2 миллионов пикселей. Говорят «2 мегапикселя». **4K** — 3840 на 2160, это уже 8 миллионов.

❓ Как думаете, почему тогда не делать все фото в огромном разрешении? *(Файл будет тяжёлым.)*

🗣️ Да: больше пикселей — картинка чётче, но и файл тяжелее, дольше грузится, занимает память.

🗣️ А теперь важная мысль. **Пиксель-арт — это когда мы специально выбираем маленькое разрешение.** Не потому что не умеем лучше, а потому что это красиво и узнаваемо. Именно это мы и будем делать в коде: уменьшать количество пикселей.

---

## Слайд 6. «Цвет: RGB»

⏱️ 4 минуты

🗣️ Помните три числа из клеточки на третьем слайде? Сейчас разберёмся, что это такое.

🗣️ Любой цвет на экране складывается из трёх цветов: **красного, зелёного и синего**. По-английски Red, Green, Blue — отсюда **RGB**. Каждый из них — число **от 0 до 255**: 0 — этого цвета нет совсем, 255 — горит на полную.

🖐️ Покажите полоски R, G, B слева.

🗣️ Смотрите: красный — 255, на полную. Зелёный — 106, чуть меньше половины. Синий — 0, выключен. Что получится? Оранжевый — вот квадратик внизу.

❓ А давайте проверим вас. Какой цвет получится, если все три числа — ноль: (0, 0, 0)? *(Чёрный.)* А если все три — 255? *(Белый.)*

🗣️ Точно. Ничего не горит — темнота, чёрный. Всё горит на полную — белый. Справа примеры: красный (255, 0, 0), жёлтый — красный плюс зелёный, (255, 255, 0).

❓ Это часто удивляет: почему жёлтый — это красный и зелёный? На уроке рисования ведь жёлтый не смешивали! *(Пусть порассуждают.)*

🗣️ Потому что на экране цвета складываются из **света**, а в красках — смешиваются **пигменты**. Это разные правила. Свет красный плюс свет зелёный — жёлтый.

🗣️ И главное число на слайде: 256 вариантов красного, умножить на 256 зелёного, на 256 синего — получается **16 777 216** цветов. Почти 17 миллионов! А в пиксель-арте обычно берут всего **16–32 цвета**. Это вторая вещь, которую мы будем делать в коде: **уменьшать количество цветов**.

> *В версии МАСТЕРСАМ можно добавить:* «Кстати, оранжевый (255, 106, 0) — это фирменный цвет нашей школы, посмотрите на логотип в углу».

---

## Слайд 7. «Соотношение сторон»

⏱️ 2 минуты

🖐️ Покажите прямоугольники.

❓ Какое соотношение сторон у видео в TikTok? *(9:16, вертикальное.)* А у YouTube? *(16:9.)*

🗣️ **Соотношение сторон** — это ширина к высоте. 16:9 — YouTube, телевизор, экран ноутбука. 4:3 — старые мониторы и многие фотоаппараты. 1:1 — квадрат, аватарки и посты. 9:16 — TikTok, Reels, Shorts: то же 16:9, только повёрнутое.

🗣️ Как его посчитать? Берём разрешение, например 1920 на 1080, и делим оба числа на одно и то же, пока можно. Оба делятся на 120: получаем 16 и 9. Значит, 16:9.

❓ Что будет, если взять фото 16:9 и сделать из него квадратную аватарку? *(Обрежется или растянется.)*

🗣️ Да: либо обрежется, либо исказится — люди станут толстыми или худыми. Это пригодится, когда будете делать пиксельную аватарку в конце занятия.

---

## Слайд 8. «Форматы и расширения»

⏱️ 3 минуты

❓ Какие расширения картинок вы знаете? То, что после точки в имени файла. *(JPG, PNG, GIF...)*

🗣️ Пробежимся по таблице.

🗣️ **JPG** — для фотографий. Файлы маленькие, но он сжимает с потерями: выкидывает детали, которые глаз почти не замечает. Если пересохранять JPG много раз, картинка портится.

🗣️ **PNG** — без потерь, и умеет быть прозрачным. **Наш пиксель-арт мы будем сохранять именно в PNG.** Догадаетесь почему? *(JPG размоет чёткие края квадратиков.)* Точно: JPG размоет границы пикселей, а нам нужны идеально чёткие квадраты.

🗣️ **GIF** — короткие анимации, мемы. Максимум 256 цветов — по сути, это тоже немножко пиксель-арт.

🗣️ **WEBP** — современный формат сайтов: лёгкий и качественный.

🗣️ **SVG** — особенный. Он вообще не из пикселей, а из формул: «нарисуй круг такого радиуса здесь». Поэтому его можно увеличивать сколько угодно — он не теряет качество. Так хранят логотипы и иконки.

🗣️ Итог внизу слайда: картинки бывают **растровые** — из пикселей: JPG, PNG, GIF, WEBP — и **векторные** — из формул, как SVG. Сегодня работаем с растровыми.

---

## Слайд 9. «Пиксель-арт в TikTok»

⏱️ 10 минут (включая просмотр роликов)

🗣️ А теперь самое интересное. Где вы видели пиксель-арт в последнее время?

*(Ждём ответы: игры, TikTok, фильтры, Minecraft, Terraria.)*

🖐️ Покажите 2–3 заранее подготовленных ролика.

🗣️ На слайде три самых популярных вида таких видео. **Пиксель-фильтр** — лицо или видео превращаются в квадратики. **8-bit версия себя** — мало цветов, выглядит как игры 90-х. И **пиксельный переход** — картинка рассыпается на блоки и собирается обратно.

❓ Посмотрите на ролики ещё раз. Что общего у всех этих эффектов? *(Мало пикселей и мало цветов.)*

🗣️ Тренды меняются каждый месяц, но секрет у всех один и тот же: **мало пикселей плюс мало цветов**. Именно эти две вещи мы и запрограммируем.

---

## Слайд 10. «Как фото становится пиксель-артом»

⏱️ 4 минуты

🗣️ Давайте превратим это в алгоритм — пошаговую инструкцию для компьютера. Посмотрите на четыре картинки.

🖐️ Ведите по картинкам слева направо.

🗣️ **Оригинал** — фото 768 на 768 пикселей.

🗣️ **Шаг первый — уменьшаем.** Делаем из фото маленькую картинку 48 на 48. Каждый квадрат 16 на 16 пикселей превращается в **одну** точку. Это ровно то, что мы видели на слайде про разрешение.

🗣️ **Шаг второй — оставляем только 32 цвета.** Вместо миллионов оттенков — 32. Компьютер сам подбирает, какие 32 цвета лучше всего описывают эту картинку.

🗣️ **Шаг третий — увеличиваем обратно, но без размытия.** Растягиваем маленькую картинку до прежнего размера так, чтобы каждая точка стала большим чётким квадратом.

❓ Подумайте: а если просто растянуть маленькую картинку обычным способом, как в любом редакторе, что будет? *(Она станет мутной, размытой.)*

🗣️ Именно. Обычное увеличение сглаживает края — и вместо пиксель-арта выходит мыло. Поэтому нужен специальный способ, без размытия. Сейчас увидим его в коде.

---

## Слайд 11. «Код на Python»

⏱️ 6 минут

🗣️ Вот вся программа. Не пугайтесь — разберём каждую строчку.

🖐️ Ведите указкой или курсором по коду.

🗣️ Первая строка: `from PIL import Image`. **Pillow** — это библиотека для работы с картинками. Мы говорим Python: «возьми из неё инструмент Image». Ставится она командой `pip install pillow`.

🗣️ Дальше открываем фото: `Image.open("photo.jpg")`. И сразу `.convert("RGB")` — переводим в те самые три числа на пиксель, о которых говорили.

🗣️ **Шаг 1** — `resize`, изменить размер. Ширину делим на 12, высоту делим на 12. Знак `//` — это деление без остатка: размер картинки должен быть целым числом. То есть каждый квадрат 12 на 12 превращается в одну точку.

🗣️ **Шаг 2** — `quantize(colors=32)`. Это и есть «оставить 32 цвета».

🗣️ **Шаг 3** — снова `resize`, но обратно до размера `photo.size`, и с волшебным словом **NEAREST** — «ближайший сосед». Этот способ просто копирует цвет каждой точки в большой квадрат и ничего не размывает. И сохраняем в PNG — помните почему?

🗣️ Справа подсказка: **числа 12 и 32 — это ваши ручки настройки.** Поменяете 12 на 30 — пиксели станут огромными. Поменяете 32 на 4 — останется всего четыре цвета. Именно с этими числами вы сейчас будете экспериментировать.

---

## Практика 1. Свой пиксель-арт

⏱️ 20 минут (слайды 11 и 14 на экране)

🗣️ Теперь ваша очередь. Есть два пути.

🗣️ Если вы раньше не программировали — откройте готовый файл `main.py` из папки проекта и запустите его. Он откроет окно выбора фото, потом спросит в терминале размер пикселя и количество цветов. Введите, например, 12 и 32. Результат появится в папке проекта: `my_pixel_art.png`, а рядом — `my_coloring.pdf`, раскраска по номерам. Её посмотрим чуть позже.

🗣️ Если вы уже писали код — создайте новый файл и наберите код со слайда сами. Положите рядом фото под именем `photo.jpg`. Так лучше запомнится.

🖐️ Ходите по классу. Типичные проблемы и решения — в конспекте, в разделе «Подготовка».

🗣️ *(Через 5–7 минут, когда у всех получилось.)* У всех появилась картинка? Отлично. Теперь поиграйте с числами: попробуйте размер пикселя 4, потом 40. Цветов — 2, потом 64. Найдите комбинацию, которая вам нравится больше всего.

🗣️ *(Опытным.)* А для вас задание: сделайте так, чтобы программа сама сохранила 5 картинок с разным размером пикселя. Подсказка — цикл `for`.

🗣️ *(В конце блока.)* Покажите соседу свой лучший вариант. Какие числа у вас вышли самыми удачными?

---

## Слайд 12. «Наша библиотека pixelkid»

⏱️ 7 минут

🗣️ Сейчас вы написали программу. А теперь представьте, что ей хочет пользоваться ваш друг. Он должен копировать все десять строк? Разбираться, что такое `quantize`?

🗣️ Программисты поступают иначе: они упаковывают код в **библиотеку**. **Библиотека — это чужой код, который подключается одной строкой.** Pillow, которым мы пользовались, — тоже чья-то библиотека.

🗣️ Так вот: мы сделали свою библиотеку и выложили её в интернет, в PyPI — это как магазин приложений, только для Python. Называется **pixelkid**. Ставится одной командой — смотрите на слайд: `pip install pixelkid`.

🖐️ Запустите `pip install pixelkid` у себя в терминале, пусть дети повторят.

🗣️ А пользоваться — вот так: `from pixelkid import pixelate`, и одна строка. Всё, что мы только что писали, спрятано внутри функции `pixelate`. Справа параметры: `pixel_size` — размер пикселя, `colors` — сколько цветов, `save_as` — куда сохранить.

🗣️ Есть секрет: внутри библиотека устроена немного умнее нашего кода на слайде. Она подбирает цвета так, как их различает человеческий глаз, и в каждую клетку ставит самый частый цвет, а не средний. Поэтому картинки у неё получаются сочнее. Кому интересно, как именно, — подойдите после занятия или загляните в README библиотеки.

🗣️ И ещё одна функция, которой нет на слайде: `coloring_pdf`. Она делает из фото раскраску по номерам.

🖐️ Покажите файл `my_coloring.pdf`, который сохранил `main.py`.

🗣️ Смотрите: на первой странице картинка разбита на клетки с номерами, внизу палитра — номер, цвет и его RGB, те самые три числа. На второй странице — ответ: как должно получиться. Раскраску можно распечатать и раскрасить дома карандашами.

---

## Слайд 13. «Пиксель-раскраска»

⏱️ 10 минут (включая практику)

🗣️ И последний шаг: из одной функции можно сделать настоящую программу с окнами и кнопками. Вот приложение «Пиксель-раскраска».

🖐️ Запустите `python app/pixel_paint.py` и откройте фото.

🗣️ Как пользоваться — на слайде. **Первое:** открываем своё фото. **Второе:** ползунками выбираем, сколько клеток и сколько цветов. **Третье:** нажимаем на цвет в палитре сверху и раскрашиваем клетки по номерам — можно вести мышкой с зажатой кнопкой. **Четвёртое:** если не уверены, зажмите кнопку «Показать подсказку» — увидите правильную картинку.

🗣️ Внизу есть кнопка «PDF-раскраска» — на скриншоте её нет, она появилась позже. Она сохраняет такой же PDF, как мы видели, с теми же номерами, что на экране.

🗣️ Внутри приложения — та же наша функция `pixelate`. А окошки, кнопки и ползунки делает другая библиотека — PyQt6. То есть настоящие программы собираются из библиотек, как из кубиков.

🖐️ Дайте детям 5–7 минут поиграть с приложением.

---

## Слайд 14. «Теперь твоя очередь»

⏱️ 3 минуты *(или фоном во время практики)*

🗣️ Вот задания, если хочется продолжить — здесь или дома.

🗣️ **Первое: поменяйте числа.** Размер пикселя от 4 до 40, цветов от 2 до 64. Найдите самую красивую комбинацию.

🗣️ **Второе: своё фото.** Селфи, питомец, любимый герой. Лучше всего получаются светлые и контрастные фото, где главный объект крупный.

🗣️ **Третье: аватарка.** Квадратная картинка 1:1 в стиле 8-bit. Вспомните слайд про соотношение сторон: фото сначала нужно обрезать до квадрата.

🗣️ **И челлендж со звёздочкой:** сделайте пять картинок с разным размером пикселя с помощью цикла `for`.

---

## Слайд 15. «Что дальше?»

⏱️ 3 минуты

🗣️ Давайте подведём итог. Посмотрите, что вы сегодня сделали. Вы узнали, что картинка — это таблица чисел. Написали программу, которая эти числа меняет. Поставили библиотеку одной командой. И поработали в приложении с окнами и кнопками.

🗣️ **Это и есть программирование**: взять задачу, разбить её на шаги и объяснить их компьютеру.

> *В версии МАСТЕРСАМ:* 🗣️ «Если вам понравилось, у нас есть курсы: Python для 12–17 лет и Python с искусственным интеллектом для 14–17 лет. Там вы будете делать игры, ботов и свои нейросети.»
>
> *В версии ШЦТ:* 🗣️ «Продолжайте изучать Python: игры, боты, нейросети — всё начинается с таких же маленьких проектов, как сегодняшний.»

🗣️ Спасибо вам! Какие есть вопросы?

*(Ответьте на вопросы. Если вопросов нет, спросите сами: «Что было самым неожиданным?», «Где ещё можно применить то, что мы сегодня сделали?».)*

---

## Частые вопросы детей и ответы

**«А можно сделать анимацию / видео?»** Можно: видео — это много картинок подряд. Если обработать каждый кадр функцией `pixelate`, получится пиксельное видео. Это хорошая тема для следующего проекта.

**«Почему у меня получилось мутно?»** Скорее всего, картинка сохранилась в JPG или при увеличении не стоит `NEAREST`. Сохраняйте в PNG.

**«Почему у моего друга цвета ярче, хотя числа те же?»** Зависит от фото: на тёмном фото все цвета тёмные. Попробуйте светлое, контрастное фото.

**«Как работает quantize?»** Компьютер ищет группы похожих цветов и заменяет каждую группу одним средним цветом. Для продвинутых: в библиотеке это делается методом k-means — подробности в `pixelkid/README.md`.

**«А нейросети так же видят картинки?»** Да, для нейросети картинка — тоже таблица чисел RGB. Всё, что вы сегодня узнали, — фундамент компьютерного зрения.